

Pokedex — consommer une API REST

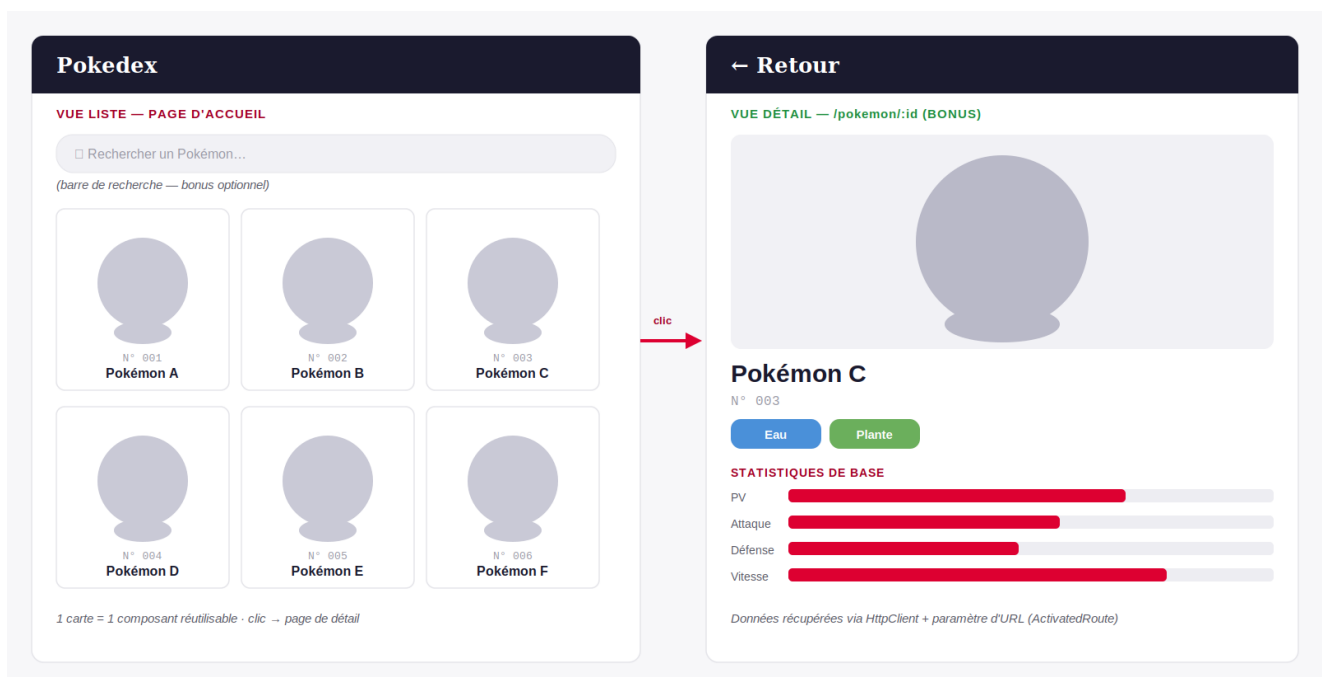
Objectif : construire seul une application Angular qui liste des Pokémon avec une recherche en direct, en consommant une vraie API publique (PokeAPI). C'est l'occasion de mobiliser, individuellement, les notions vues depuis le début de la semaine : composants, services, HttpClient. La page de détail (avec routage) est proposée en bonus pour ceux qui veulent aller plus loin.

Travail strictement individuel — pas d'échange de code entre étudiants. Vous pouvez bien sûr consulter la documentation Angular et PokeAPI. Le délai de rendu vous sera communiqué séparément.

CONSIGNES OBLIGATOIRES

- **1. Liste des Pokémon** — Appeler l'API PokeAPI via un service Angular dédié (HttpClient), afficher les résultats sous forme de cartes (un composant par carte), avec une image (voir l'indice sprite plus bas)
- **2. Recherche par nom** — Filtrer la liste en direct pendant que l'utilisateur tape (voir l'indice RxJS plus bas)
- **3. Architecture propre** — Séparer clairement service (appels API), composants (affichage), et modèle (type TypeScript des données reçues)

MAQUETTE ATTENDUE (structure — pas le design final)



Les silhouettes grises sont volontairement génériques — libre à vous d'utiliser les vraies images renvoyées par PokeAPI. Seule la vue liste (à gauche) est imposée ; la vue détail (à droite, avec le clic et le routage) est un bonus. La mise en forme (couleurs, disposition) reste à votre appréciation.

■ **Indication — barres de statistiques :** pas besoin d'une librairie de graphiques (type Chart.js) pour afficher les statistiques sous forme de barres. Une simple div CSS dont la largeur est calculée en pourcentage suffit largement :

```
<div class="stat-bar-bg">
  <div class="stat-bar-fill"
    [style.width.%]="(stat.base_stat / 255) * 100">
  </div>
</div>
```

`[style.width.%]` écrit directement une propriété CSS depuis le TypeScript. `255` est la valeur maximale théorique d'une statistique Pokémon — le calcul donne donc un pourcentage cohérent, sans rien d'autre à configurer.

■ **Indication — la réponse détail de l'API est volumineuse** : en appelant `/pokemon/{id}`, vous recevrez bien plus de champs que nécessaire (capacités, historique de jeux, toutes les variantes de sprites par génération...). Concentrez-vous sur : un identifiant (`id`), un nom (`name`), une image (du côté de `sprites`), une liste de types (`types`, avec un nom par entrée), et une liste de statistiques (`stats`, chaque entrée ayant une valeur et un nom). Ne tapez dans votre interface TypeScript que les champs que vous utilisez réellement — TypeScript ne se plaint pas des champs en trop non déclarés, il ignore simplement le reste.

■ **Indication — afficher une image sur chaque carte de la liste sans multiplier les appels** : la liste (`/pokemon?limit=20`) ne renvoie qu'un nom et une URL par Pokémon — pas d'image. Un réflexe naturel serait de faire un appel détaillé supplémentaire pour chaque carte (20 requêtes en plus), mais ce n'est pas nécessaire. L'identifiant de chaque Pokémon peut être extrait directement de l'URL renvoyée par la liste (le dernier segment de l'URL est l'id). Les images suivent ensuite un nommage prévisible sur le dépôt GitHub officiel de PokeAPI :

```
https://raw.githubusercontent.com/PokeAPI/sprites
/master/sprites/pokemon/{id}.png
```

Il suffit de remplacer `{id}` par l'identifiant extrait de l'URL pour obtenir directement l'image — un seul appel HTTP suffit alors pour toute la liste avec images. (Si votre réseau bloque ce domaine, un miroir équivalent existe sur `cdn.jsdelivr.net/gh/PokeAPI/sprites/sprites/pokemon/{id}.png`.)

■ **Indication — l'affichage ne se met pas à jour après un appel API** : les projets Angular récents fonctionnent en mode *zoneless* par défaut. Une simple propriété de classe (`loading = true`) ne suffit plus à prévenir automatiquement le template quand sa valeur change après une opération asynchrone (comme une requête HTTP) — contrairement à ce qui se passait avec Zone.js. Toute variable lue dans une condition d'affichage (`@if`, `@else if`) doit être déclarée avec `signal(...)`, et mise à jour avec `.set(...)` plutôt qu'une simple affectation. Repérez d'abord toutes les variables utilisées dans vos blocs `@if` du template, puis vérifiez qu'elles sont bien déclarées comme des signals dans le composant — **toutes**, pas seulement une au choix.

BONUS (facultatif, pour aller plus loin)

- Page de détail par Pokémon (Angular Router + paramètre d'URL), affichant nom, image, types, statistiques de base
- Indicateur de chargement (spinner ou message) pendant l'appel API
- Gestion d'une erreur réseau (message affiché si l'API ne répond pas)

Hors scope : pas de création, modification ou suppression de Pokémon — PokeAPI est en lecture seule (GET uniquement). Contrairement au projet de groupe, aucun CRUD complet n'est attendu ici.

API À UTILISER — PokeAPI (gratuite, sans clé)

Liste des Pokémon	https://pokeapi.co/api/v2/pokemon?limit=20
Détail d'un Pokémon	https://pokeapi.co/api/v2/pokemon/{nom-ou-id}
Documentation	https://pokeapi.co/docs/v2

RENDU ATTENDU

- **Projet complet** déposé sur la plateforme indiquée par l'école — code source dans son intégralité, pas un extrait
- **Un fichier README.md à la racine du projet**, contenant obligatoirement :
 - Votre nom et prénom
 - Une courte description du projet
 - Les instructions pour installer et lancer l'application (npm install, ng serve...)
 - Les choix techniques ou difficultés rencontrées, en quelques lignes
- Application fonctionnelle, lancée avec ng serve, sans erreur bloquante dans la console
- Pas de soutenance à l'oral pour ce TD — évaluation sur le rendu et le code uniquement
- **Délai de rendu : communiqué séparément** — ce document ne fixe ni date ni heure

CRITÈRES D'ÉVALUATION

- Les 3 consignes obligatoires sont fonctionnelles (liste, recherche, architecture)
- Le service est correctement séparé des composants (pas d'appel HttpClient directement dans un composant)
- Les données reçues de l'API sont typées (interface TypeScript), pas de any
- Code lisible : noms de variables clairs, pas de duplication évidente
- README.md complet et clair : un correcteur doit pouvoir lancer le projet sans vous poser de question
- Bonus : page de détail avec routage fonctionnel, si tentée